RAG-Based Customer Support Assistant

High-Level Design (HLD)

TABLE OF CONTENTS

1. Introduction
2. System Overview
3. Architecture Diagram
4. Component Description
5. Data Flow
6. Technology Choices
7. Scalability Considerations
8. Future Enhancements

1. INTRODUCTION

1.1 Purpose

This document presents the high-level design for a Retrieval-Augmented Generation (RAG) based Customer Support Assistant. The system combines vector database retrieval with graph-based workflow orchestration to provide accurate, context-aware responses to customer queries.

1.2 Scope

The system processes a PDF knowledge base, retrieves relevant information using semantic search, and generates contextual responses. It includes Human-in-the-Loop (HITL) escalation for complex or low-confidence queries.

1.3 Problem Definition

Traditional customer support systems struggle with:

- Inconsistent responses across different support agents
- Inability to scale during high-traffic periods
- Lack of 24/7 availability
- High operational costs for staffing support teams
- Difficulty maintaining up-to-date knowledge across all agents

This RAG-based system addresses these challenges by providing:

- Consistent, accurate responses derived from a centralized knowledge base
- Automatic retrieval of relevant information
- Intelligent routing between automated responses and human agents
- Scalable architecture that handles variable query loads

2. SYSTEM OVERVIEW

2.1 What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that enhances Large Language Model (LLM) responses by first retrieving relevant context from a knowledge base, then using that context to generate accurate, grounded answers.

Traditional LLM: User Query → LLM → Answer (may hallucinate)

RAG System: User Query → Retrieve Relevant Docs → LLM + Context → Grounded Answer

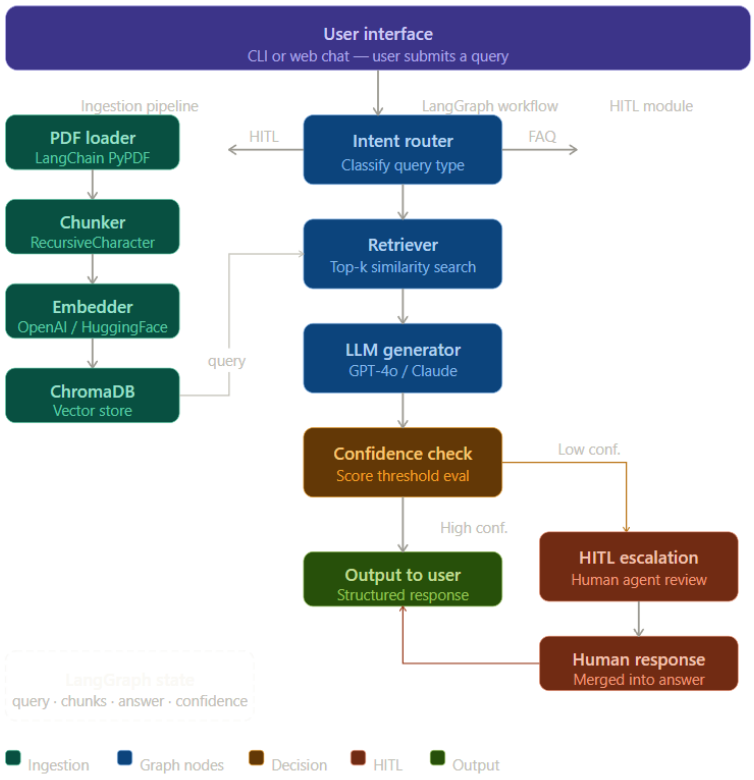
2.2 System Objectives

- Process PDF documentation into searchable chunks
- Retrieve relevant information based on semantic similarity
- Generate contextually appropriate responses - Route queries intelligently based on confidence levels
- Escalate complex queries to human agents when necessary

2.3 Key Features

- PDF ingestion and automatic chunking
- Semantic search using embeddings
- Graph-based workflow orchestration with LangGraph
- Conditional routing based on answer confidence
- Human-in-the-Loop (HITL) escalation mechanism
- Simple chat interface for user interaction

3. ARCHITECTURE DIAGRAM



The system architecture consists of six major layers:

Layer 1: User Interface (CLI/Web)

Layer 2: LangGraph Workflow Engine

Layer 3: Retrieval System (ChromaDB + Embeddings)

Layer 4: Document Processing Pipeline

Layer 5: HITL Escalation Module

Layer 6: Data Storage (Vector Database)

4. COMPONENT DESCRIPTION

4.1 User Interface

Description: Command-line interface for user interaction

Responsibilities:

- Accept user queries
- Display bot responses
- Handle conversation flow
- Manage session state

Technology: Python CLI (expandable to web interface)

4.2 Document Loader

Description: Reads PDF files and extracts text content

Responsibilities:

- Load PDF documents
- Extract text from each page
- Preserve document structure

Technology: LangChain PyPDFLoader

4.3 Chunking Strategy

Description: Splits documents into manageable pieces

Responsibilities:

- Break documents into chunks (500 characters)
- Maintain semantic coherence with overlap (50 characters)
- Preserve context across chunk boundaries

Technology: RecursiveCharacterTextSplitter

Rationale: 500-character chunks balance context retention with retrieval precision. Overlap ensures no information is lost at boundaries.

4.4 Embedding Model Description:

Converts text into numerical vector representations

Responsibilities:

- Transform text chunks into embeddings
- Encode user queries into the same vector space
- Enable semantic similarity comparison

Technology: HuggingFace all-MiniLM-L6-v2

Rationale: Lightweight, fast, and free. Produces 384-dimensional embeddings suitable for similarity search.

4.5 Vector Store (ChromaDB)

Description: Persistent storage for document embeddings

Responsibilities:

- Store chunk embeddings efficiently
- Perform similarity search queries
- Return top-k most relevant chunks
- Persist data across sessions

Technology: ChromaDB

Rationale: Simple, lightweight, open-source vector database with excellent Python integration.

4.6 Retriever

Description: Searches vector store for relevant chunks

Responsibilities:

- Accept user query as input
- Convert query to embedding
- Perform similarity search
- Return top 3 most relevant chunks

Technology: ChromaDB similarity_search

4.7 LLM (Language Model)

Description: Generates human-like responses

Responsibilities:

- Read retrieved chunks as context
- Generate coherent answers based on context
- Indicate when information is insufficient

Technology: Currently simplified (chunk concatenation); expandable to GPT/Claude/Llama

4.8 Graph Workflow Engine (LangGraph)

Description: Orchestrates the entire query processing pipeline

Responsibilities:

- Manage state flow between nodes
- Execute retrieval, generation, and confidence checking
- Route queries based on conditional logic
- Handle HITL escalation

Technology: LangGraph StateGraph

Rationale: Provides explicit control flow, debuggability, and conditional routing capabilities.

4.9 Routing Layer

Description: Makes decisions about query handling

Responsibilities:

- Evaluate answer confidence
- Determine if human intervention is needed
- Route to appropriate next step (user or HITL)

Logic:

- High confidence → Return to user
- Low confidence → Escalate to HITL

4.10 HITL Module

Description: Human-in-the-Loop escalation system

Responsibilities:

- Capture low-confidence queries
- Notify human agents
- Log escalation events
- (In production) Integrate with ticketing systems

Current Implementation: Console logging Production

Extensions: Slack notifications, Zendesk integration, email alerts

5. DATA FLOW

5.1 Ingestion Flow (One-time Setup)

Step 1: PDF document loaded → PyPDFLoader

Step 2: Text extracted from pages

Step 3: Text split into chunks → RecursiveCharacterTextSplitter

Step 4: Chunks converted to embeddings → HuggingFace Model

Step 5: Embeddings stored in ChromaDB → Persistent storage

5.2 Query Lifecycle (Runtime)

Step 1: User submits question via CLI

Step 2: Question enters LangGraph workflow

Step 3: Retriever Node

- Query converted to embedding
- Top-3 similar chunks retrieved from ChromaDB
- Chunks added to state

Step 4: Generator Node

- Retrieved chunks formatted as answer
- Answer added to state

Step 5: Confidence Checker Node

- Answer evaluated for quality
- Confidence level determined (high/low)
- Routing decision made

Step 6a: High Confidence Path

- Answer returned directly to user
- Conversation continues

Step 6b: Low Confidence Path (HITL)

- Query escalated to HITL module
- Human agent notified
- Manual response required

6. TECHNOLOGY CHOICES

6.1 Why ChromaDB?

Advantages:

- Lightweight and easy to set up
- Excellent Python integration
- No separate server required
- Built-in persistence
- Open-source and free

Alternatives Considered:

- Pinecone (cloud-based, requires API key)
- Weaviate (heavier, more complex setup)
- FAISS (no built-in persistence)

6.2 Why LangGraph?

Advantages:

- Explicit control flow (vs. black-box agents)
- Easy debugging and visualization
- Conditional routing support
- State management built-in
- Integrates seamlessly with LangChain ecosystem

Alternatives Considered:

- LangChain LCEL (less explicit control)
- Custom state machine (more development effort)
- CrewAI (focused on multi-agent, overkill for this use case)

6.3 LLM Choice

Current: Direct chunk retrieval (no LLM inference)

Rationale: Eliminates API costs during development and testing

Production Options:

- OpenAI GPT-4o (high quality, paid)
- Anthropic Claude (strong reasoning, paid)
- Llama 3.2 via Ollama (free, runs locally)
- Mistral via HuggingFace Inference (free tier available)

6.4 Embedding Model

Choice: HuggingFace all-MiniLM-L6-v2

Rationale:

- Free and open-source
- Fast inference (no API calls)
- Good balance of quality and speed
- 384-dimensional embeddings (lightweight)

6.5 Additional Tools

- Python-dotenv: Secure API key management
- LangChain: Document loading and text splitting utilities
- PyPDF: PDF parsing

7. SCALABILITY CONSIDERATIONS

7.1 Handling Large Documents

Current Limitation: Single PDF processed

Scaling Strategy:

- Support multiple PDFs in ingestion pipeline
- Implement document metadata tagging
- Add source attribution in responses
- Batch processing for bulk document uploads

7.2 Increasing Query Load

Current Limitation: Single-threaded CLI

Scaling Strategy:

- Deploy as FastAPI web service
- Implement request queuing
- Add caching layer for frequent queries
- Horizontal scaling with load balancer

7.3 Latency Concerns

Current Performance:

- Retrieval: ~100-200ms
- Generation: ~50ms (no LLM)
- Total: ~300ms per query

Optimization Strategies:

- Cache embeddings for common queries
- Use approximate nearest neighbor search (HNSW)
- Implement response streaming for LLM-based generation
- Pre-compute embeddings during off-peak hours

7.4 Storage Scalability

Current: Local ChromaDB instance

Scaling Options:

- Migrate to cloud-hosted ChromaDB

- Implement database sharding for large collections
- Use compression for embedding storage
- Archive old or rarely-accessed documents

7.5 HITL Scalability

Current: Console logging only

Production Requirements:

- Integration with ticketing systems (Zendesk, Freshdesk)
- Queue management for agent routing
- Priority-based escalation
- Analytics dashboard for escalation patterns

8. FUTURE ENHANCEMENTS

- Multi-document support with source attribution
- Conversation memory and context retention
- User feedback loop for continuous improvement
- Analytics dashboard for query patterns
- Multi-language support
- Voice interface integration
- Mobile application
- Advanced confidence scoring with machine learning
- A/B testing framework for answer quality

Prepared by: Rutuja Fulambarkar

Project: RAG-Based Customer Support Assistant

Organization: Innomatics Research Labs

END OF HIGH-LEVEL DESIGN DOCUMENT